

118. enum : un nom pour chaque écran

Un jeu passe d'écran en écran : titre, partie, perdu. Une variable dit où l'on en est, et chaque valeur a un nom.

1. Ce qu'il faut comprendre

Un vrai jeu n'est pas une seule boucle qui fait toujours la même chose. Il y a **l'écran titre**, qui attend START ; **la partie** ; **l'écran perdu**, qui attend qu'on recommence. Les mêmes boutons n'y font pas la même chose : START lance la partie au titre, et ramène au titre quand on a perdu.

On range donc, dans **une variable**, l'écran où l'on est : son **état**. On pourrait écrire 0 pour le titre, 1 pour la partie, 2 pour perdu. Mais `if (etat == 2)`, dans trois semaines, ne dira plus rien à personne.

Ce qui est nouveau ici : `enum`. `enum Etat { TITRE, JEU, PERDU };` crée **trois noms** pour les nombres 0, 1 et 2, dans l'ordre, et un **type** `Etat`. `Etat etat = TITRE;` déclare une variable de ce type. Pour la console, c'est un octet ; pour qui lit, `if (etat == PERDU)` se comprend tout seul.

La boucle regarde l'état et ne fait **que ce qui le concerne** : au titre, elle attend START ; en jeu, A fait perdre (en attendant un vrai jeu) ; perdu, START ramène au titre.

Le front de START (chapitre 11) est indispensable : sans lui, un appui de quelques images ferait TITRE → JEU, puis, dès qu'on perd, PERDU → TITRE → JEU d'un seul coup, trop vite pour être vu.

`dessine` retient l'état **déjà montré** à l'écran. Quand `etat` change, les deux diffèrent : on redessine **une fois**, puis `dessine = etat`. C'est l'idée de toujours : n'écrire que ce qui change.

À toi : ajoute un état `PAUSE`, où l'on entre et d'où l'on sort avec SELECT pendant la partie.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **837 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	59 o	← la console, à l'allumage — la mise en route de la console
main()	221 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte — attend de pouvoir écrire une case
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
CopierTuiles	76 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
LireManette	37 o	← bouton() — lit les huit boutons

```

Tuiles          256 o ← CopierTuiles - 16 lettres sur 43
"              ..." 20 o ← texte() - un texte gravé
"              ..." 20 o ← texte() - un texte gravé
"MON JEU"       7 o ← texte() - un texte gravé
"START: JOUER"  12 o ← texte() - un texte gravé
"LA PARTIE"     9 o ← texte() - un texte gravé
"A: FAIRE PERDRE" 15 o ← texte() - un texte gravé
"PERDU"         5 o ← texte() - un texte gravé
"START: TITRE"  12 o ← texte() - un texte gravé

```

Lettres gravées : ADEFIJLMNOPRSTU:

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

```

texte() × 8    88 o sur place, dans main() + EcrireTexte (10 o)
bouton() × 2   17 o sur place, dans main() + LireManette (37 o)
image() × 1    3 o sur place, dans main() + AttendreImage (28 o)

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// CE PROGRAMME : trois écrans : le TITRE (START pour jouer), la PARTIE
// (A pour perdre), et PERDU (START pour revenir au titre).
//
// Ce qui est nouveau : enum, des NOMS pour des nombres.
//  enum Etat ... = on invente un type « Etat » dont les valeurs ont des noms.
//  TITRE vaut 0, JEU vaut 1, PERDU vaut 2 – mais on n'a plus à s'en souvenir.
//  « etat == PERDU » se lit mieux que « etat == 2 ».

// Les trois écrans du jeu : TITRE vaut 0, JEU vaut 1, PERDU vaut 2.

#include <bouton> // lit un bouton de la manette
#include <texte>  // écrit un texte à l'écran

```

2. DES CONSTANTES NOMMÉES — ETAT

```
enum Etat { TITRE, JEU, PERDU };
```

3. UNE VARIABLE — ETAT

```
Etat etat = TITRE; // où l'on en est : on commence au titre
```

4. UNE VARIABLE — DESSINE

```
uint8_t dessine = 255; // l'état déjà montré : 255, aucun, pour dessiner au début
```

5. UNE VARIABLE — AVANT

```
uint8_t avant = 0; // START à l'image d'avant
```

6. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                // Le jeu commence ici.
    while (true) {                          // La boucle du jeu :
        image();

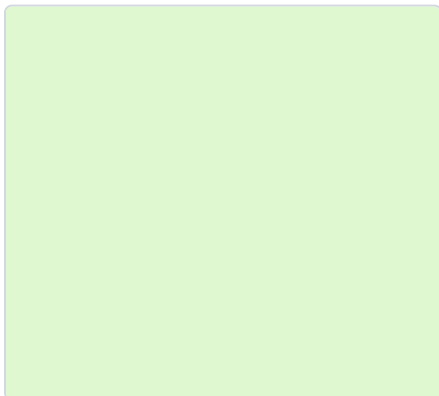
        // Le front de START : enfoncé maintenant, pas à l'image d'avant.
        uint8_t start = bouton(START);
        uint8_t appui = start && !avant;    // 1 seulement à la première image de l'appui
        avant = start;

        // Chaque état ne fait que ce qui le concerne.
        if (etat == TITRE) {               // Au titre :
            if (appui) etat = JEU;          // START → la partie.
        } else if (etat == JEU) {          // Pendant la partie :
            if (bouton(A)) etat = PERDU;    // A → perdu.
        } else if (etat == PERDU) {        // À l'écran perdu :
            if (appui) etat = TITRE;        // START → retour au titre.
        }

        // L'état a changé ? On redessine l'écran, une seule fois.
        if (etat != dessine) {
            dessine = etat;                 // On retient ce qu'on montre.
            texte(0, 8, "                  "); // Efface les deux lignes de texte
            texte(0, 10, "                ");  // (20 espaces chacune).
            if (etat == TITRE) {
                texte(6, 8, "MON JEU");
                texte(4, 10, "START: JOUER");
            } else if (etat == JEU) {
                texte(5, 8, "LA PARTIE");
                texte(3, 10, "A: FAIRE PERDRE");
            } else {                        // le seul autre cas : PERDU
                texte(7, 8, "PERDU");
                texte(4, 10, "START: TITRE");
            }
        }
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



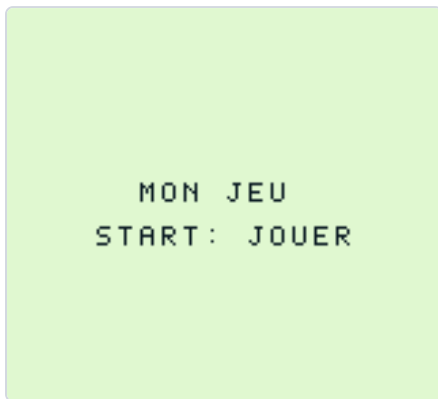
la toute première image — la console vient de démarrer — image 1



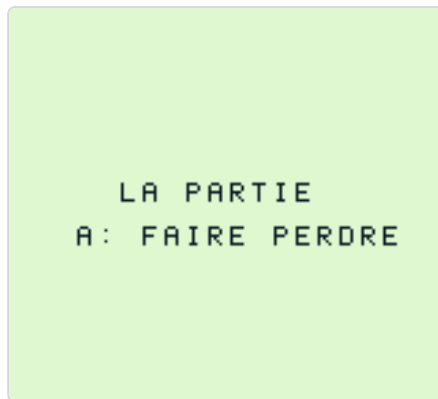
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



A — l'écran n'a pas bougé



START — l'écran a changé

5. Ce qu'on doit voir

MON JEU ; START passe à LA PARTIE ; A affiche PERDU ; START ramène au titre.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ au départ, l'écran titre : état vaut TITRE (0)
- ✓ START lance la partie : état vaut JEU (1)
- ✓ A fait perdre : PERDU (2)
- ✓ START ramène au titre, et pas plus loin

Ensuite : **119. Un switch, et une fonction par état** — chapitre 20 : Les états du jeu : titre, partie, fin. · gameboy3 — écrire une cartouche Game Boy en C++